Preserving Execution Constraints in a Composable Agent Runtime

OpenBot

***Abstract*—An agent runtime must combine tools, limits, prompt changes, and result hooks without losing the rules that each part needs. A common interface does not give this guarantee. Two well-typed plugins can replace a tool, change the meaning of a check, or repeat an outside write. This design and analysis paper defines a typed registration protocol and studies that gap. Right-biased tool maps, short-circuit checks, prompt functions, and resource scopes have different composition laws. We state conditions for associative grouping, denial before model and tool work, and resource ownership. We also show why idempotent hooks alone do not make a sequence safe to replay. A validation method combines type checks, registration checks, trace tests, and small proof obligations. We prove a sufficient law for replay-safe hook composition and state the transaction assumptions for at most one local write per key. A finite experiment checks 1,472 base traces and 3,182 extended traces against a separate leaf oracle. It also checks all 1,681 ordered pairs of idempotent functions on four states and all 4,166 finalizer dependency graphs through four resources. Six single-rule ablations expose six designed faults. These are measurements of a finite model, not evidence of production conformance or performance. The scope is plugin and runtime composition; full workflow recovery and access-control policy are outside it.**

***Index Terms*—Agent runtime, Plugin composition, Effect types, Structured concurrency**

I. Introduction

An agent runtime combines model calls, tools, limits, prompt filters, and resource connections. Each part may work alone. The hard question is whether its rules survive composition. Every relevant path must pass a limiter's check. A tool definition must stay paired with the right handler. A connection can close only after its users stop.

Stream wrappers can choose when to call the next wrapper, repeat it, or handle its failure. This hides control flow inside feature code. Explicit registration offers a smaller interface: plugins provide tools and named hooks, and the host calls them at fixed points. Yet hooks still have order, effects, failures, and shared state. A record interface does not remove these facts.

We propose a runtime in which plugins build registrations in a scope: a lifetime that owns resources and their release actions. The host builds these records and owns the step loop. A defined weaker baseline shares the field types but lacks checks on replacement, prompt structure, and replay. Both designs are specified here.

This paper makes three contributions. First, it gives a small algebra for the defined registration interface. It proves grouping laws under stated assumptions and identifies operations that are sensitive to order. Second, it gives validation obligations for preserving execution constraints: complete dispatch through the host, paired tool replacement, stable scope ownership, prompt contracts, and safe result-hook replay. Third, it gives a reproducible finite experiment with an independent oracle, complete bounded enumeration, and single-rule ablations. It separates these measured results from a further plan for compiler and runtime studies.

The results are proof arguments, worked counterexamples, and a finite model experiment. They are not a verified implementation or a study of production workloads. Plugins are cooperating modules, not hostile code confined by a sandbox. The protocol is language independent and permits modeled foreign calls.

II. Foundations and related work

Plotkin and Pretnar give a theory in which handlers interpret operations for computational effects [1]. Our protocol makes a related separation between declared tools and host execution. Its hooks are typed computations with explicit sequencing, not proved instances of that formal handler calculus. The separation guides the design without supplying its soundness proof. Wu, Schrijvers, and Hinze show that handler order changes effect interaction, and separate the syntax of scopes from their interpretation [2]. Fixed host phases make a different separation: they decide when hooks run. They do not make the hooks commute.

Hyland, Plotkin, and Power distinguish combination from commutative combination through sum and tensor constructions [3]. A right-biased map merge and sequential hook composition can likewise be associative without allowing plugins to be exchanged. Our algebra is a record algebra, not their tensor construction.

Leijen's Koka work gives effect types a formal relation to program behavior [4]. In that system, the absence of an exception effect has a proved meaning. Our proposed interface tracks successful values, typed failures, and required services. Those channels help preserve dependencies through composition. They do not, by themselves, express an upper bound on tool calls, a stable replay key, or the meaning of a prompt filter. Foreign code also lies outside a local type proof unless its behavior is modeled. We therefore distinguish a well-typed interface from a behavioral guarantee. Tang and Lindley give type- and semantics-preserving translations from row-based and capability-based effect systems into a common modal framework [5]. No such translation is proved for this runtime. A service name in our interface is not by itself a capability proof.

Structured concurrency places concurrent work inside a parent lifetime. Swift's primary design proposal specifies

child-task and task-group lifetime rules, while treating cancellation as cooperative [6]. Our scope protocol uses owned tasks for a related purpose. The useful common rule is that work has an owner. Neither approach makes an outside write reversible merely because its local task is cancelled. Matache and colleagues give a sound and complete equational treatment of scoped effects, with scopes treated as resources [7]. Our resource ledger has an operational purpose: it checks acquisition, use, and release order. It does not inherit that equational theory or solve the general combination of scoped effects.

Kragl, Qadeer, and Henzinger develop modular refinement proofs for structured concurrent programs [8]. Refinement means preserving a simpler model's required behavior. Their method uses explicit obligations and invariants. Our proposal is not a verified translation into their system. It shares the need to state each local proof's assumptions. A hook's guarantee requires later hooks to preserve those assumptions.

OpenAPI describes HTTP interfaces in a form that tools and people can read [9]. It helps keep schemas and generated clients aligned. It does not express all temporal facts, such as a hook running after saved output but before a turn ends. We treat contract generation as one layer of composition checks and use trace properties for rules across time.

III. Problem model and assumptions

A turn is one accepted unit of conversation work. A step is a host-controlled phase within that turn. A step may call a model or carry out a host action. A registration is written

$$R = (T, H, I, B, A, F, D). \tag{1}$$

Here T maps tool names to definitions. Each definition contains input and output schemas, annotations, and a handler key. H maps those keys to handlers. I is an ordered prompt of instructions. B is a check before the step. A is a hook after saved output. F changes the prompt shown to the model. D is an optional duration and error constructor. A plugin has a stable identity and a build computation that returns a registration. It may also declare stores under owned names.

Write `Comp<V, E, S>` for a computation that succeeds with a value of type `V`, fails with a value of type `E`, or uses services named by `S`. Interruption and defects are separate exits that the host preserves. Sequential bind runs the second computation only after success of the first. Its failure and service sets are the unions of those of both parts. This is a specified interface discipline; no soundness of a particular compiler is assumed.

A step input is a record containing a turn ID, positive step number, kind (`model` or `action`), saved history, and configuration version. A result adds saved output and a finish reason. The check, result hook, prompt function, and construction function have these types:

```
check: Input -> Comp<Bool, E, S>
after: Result -> Comp<Unit, E, S>
prompt: Prompt -> Comp<Prompt, E, S>
build: Scope ->
  Comp<Registration,
       BuildError, BuildServices>
```

The different channels prevent a build failure from being confused with an allowed step, but cannot prove a hook's meaning.

The test trace records build, acquire, check, transform, model, tool, save, hook, release, and end events. Observational equivalence means equality of these ordered events, including selected tools, visible values, and failures, for the same input and matching environment replies. We erase only internal allocation and grouping nodes. A divergent run is compared by all of its finite event prefixes; equality of terminating outputs alone is insufficient. For a nondeterministic environment, compare sets of traces under the same allowed replies. This does not claim equal probabilities or equal elapsed time. Omitting any further event order needs an explicit contract.

A host state contains its phase, current leaf position, saved step, scope state, live-resource ledger, and receipt ledger. A check transition with result true advances the leaf position. A false result or an error moves to close. Only a completed true check chain can move to prepare and dispatch. A release transition removes one live resource after its release action ends. A blocked join or release keeps the scope in closing. These transition conditions define the safety properties below; termination is a separate assumption.

We assume ordinary immutable registration records. Reading a field must not run an outside effect. The host is trusted to own the dispatch path. Plugins must not call the model or a tool behind that path. Service providers must meet their declared interfaces. Scope closure must be awaited, and native adapters must cooperate with interruption. Time-limit claims assume a common saved start and matching logical clock reads at matching phases. Real wall-clock scheduling can give different boundary outcomes, so common start time alone is insufficient for trace equality. They are admission and interruption claims, not hard real-time bounds on remote work.

The constraints are distinct: denial blocks step dispatch; selected definitions match handlers; resource owners outlive uses; result hooks tolerate replay; prompt functions preserve required invariants; and failures retain diagnostic causes.

These constraints do not establish a general access-control algorithm. Nor do they prove recovery of every workflow or exactly-once behavior for arbitrary outside services. They define the seam where a runtime combines plugins while preserving local execution rules.

IV. An ordered registration algebra

Let $L \diamondsuit R$ mean merge left registration L with right registration R. Figure 1 defines the merge. A missing check means true. A missing result hook does nothing. A missing prompt function is the identity function. These defaults give an empty registration its identity behavior. The symbol `;` means sequential bind with failure propagation. `append` joins ordered sequences. `override` selects the right value on each shared key.

```
merge(L, R):
  T = override(L.T, R.T)
  H = override(L.H, R.H)
  I = append(L.I, R.I)
  B(x) = L.B(x); allowed ->
           if allowed then R.B(x) else false
  A(y) = L.A(y); R.A(y)
  F(p) = L.F(p); changed -> R.F(changed)
  D = shorter(L.D, R.D)
  return (T, H, I, B, A, F, D)

shorter(a, b):
  if a is absent: return b
  if b is absent: return a
  if a.duration <= b.duration: return a
  return b
```

Figure 1. Defined merge rules. Equal durations select the left error constructor.

For a tool key k, right-biased union chooses $R.T(k)$ when it is present and otherwise chooses $L.T(k)$. Handler maps use the same direction of precedence. Each tool in a well-formed leaf registration has one handler key, and distinct tool names have distinct keys. The validator checks these facts again after merge. It records a plugin-and-version token on both each definition and its handler. A selected pair must have equal tokens and compatible schemas. Map equality treats entry order as unobservable. The order of tool calls comes from the saved call list, not the map representation. Without these conditions, consistent merge direction alone cannot prevent a shared handler key from changing another tool.

The weak baseline uses Figure 1 without those checks and accepts all collisions. The proposed validator rejects an undeclared collision. It accepts a replacement only when its declaration names the replaced tool and version and passes schema and behavior review. Intentional replacement can be useful. The policy must be explicit rather than an accidental result of map merge.

Claim 1: grouping preserves registration behavior under fixed order. Assume the observation model above, associative sequential bind after erasing grouping nodes, and matching primitive replies and logical clock reads. Then $(L \diamondsuit R) \diamondsuit S$ and $L \diamondsuit (R \diamondsuit S)$ have the same field behavior. For maps, each key selects its rightmost occurrence in the ordered triple. For instructions, both sides append the same three sequences. For result hooks and prompt functions, both sides run the same functions in the same order. For checks, both evaluate the same prefix up to the first false result or failure. The selected deadline is the leftmost member among the minimum-duration entries, in either grouping. These arguments also give identity laws for an empty registration. Validation runs once on the complete flattened leaf list, in fixed leaf order, so its decision is also independent of grouping. Validation at an intermediate tree node is excluded: an early failure could prevent later build effects in only one grouping. The argument is a fieldwise proof for the defined algebra, not a theorem about arbitrary effect interpreters.

Reordering is different. Exchanging two definitions of `send` changes the selected handler. Exchanging a marker-removal function and a marker-expansion function can change model input. Equal durations can have different error constructors, so complete deadline values need not commute.

Plugin construction adds another obligation. Define construction of a composite as building its left child and then its right child in the same scope, followed by Figure 1. Reassociation preserves the leaf build order. It does not make build effects pure. If the second build fails, the first may already have opened a connection. Correct scope closure must release that connection. It cannot erase an outside message sent during construction.

Store composition is partial: a merge may be undefined for conflicting names. Define it as disjoint map union, with an error if the two key sets overlap. For pairwise disjoint names, grouping gives the same declarations. With a duplicate, both groupings are rejected. This gives rejection equivalence only. Full trace equality also needs a common diagnostic rule, such as reporting the first conflict in the flattened leaf order. Otherwise the first reported conflict can differ. Composite identity is the ordered list of leaf identities, not a new store owner. Tool merge alone carries no implicit store behavior.

Commutativity can be admitted only under a stronger contract. Tool and handler keys must be disjoint or equivalent. Instructions and prompt changes must commute under the chosen observation. Hook effects must not interfere. Checks must be pure predicates on the same state if their order is to be ignored, and failure selection must also be unobservable or equivalent. These are useful proof obligations for a special pair of plugins. They are not a default property of the framework.

V. Preserving the step boundary

The host has six phases: construct, check, prepare input, execute, run result hooks, and close. Construction occurs once per turn attempt. Repeated steps cycle from check through hooks; closure follows the attempt's exit. After construction, selection removes tools outside the chosen mode. The check runs B on step input. False ends the turn; failure propagates its cause. Both reach closure. Model steps and direct actions use the same check path.

At check and before dispatch, compare the clock with saved start plus D's duration. Expiry raises its error. An owned timer requests cancellation at the same deadline during execution. A retry does not renew this budget.

After a true result, input preparation applies leaf prompt functions in order, checking each declared invariant before the next function. It uses previously saved model input unchanged if one exists. Otherwise it saves the prepared input before execution. Execution saves complete model output before dispatching new local calls. It saves each final tool result before publishing it. Result hooks receive saved values in registration order. The step completes only after every hook succeeds. Hook failure leaves it open for another attempt.

Claim 2: a denied check blocks host dispatch. Let a chain have checks b_1 through b_n. Assume all model and local tool work for this step is downstream of the chain. If b_j is the first check that returns false, no check with index greater than j runs and no downstream model or tool call begins. The proof is by induction on Figure 1. The true branch continues to the suffix. The false branch returns false without evaluating it. The host then exits to closure. The same reasoning applies to a failure that exits the chain before dispatch.

A separate monotonicity law holds only for pure total checks on the same immutable input: the set of allowed inputs is the intersection of the sets allowed by each check. Adding a check can then only reduce that set. This law fails for effectful checks. If a check reads a false flag, it denies. An added earlier check can set that flag to true and then allow, making the chain allow. Claim 2 still holds because it concerns an actual denial, not extension of the plugin list.

The assumptions give this claim its value and its limits. It does not stop effects already done during registration or by earlier checks. It does not stop a remote action accepted during an earlier step. It also does not promise termination if a check never returns. A check may read current state, but a later check may change that state before dispatch. If the property must still hold then, the later change needs a preservation proof or the host needs an atomic validation at the relevant boundary.

Prompt processing creates a related hazard. The guard sees a history before prompt transformation. It cannot infer a semantic property of a later prompt merely because it approved the earlier one. A transform must preserve that property, or a second check must validate the actual model input. In the latter case, that second check also precedes model and tool dispatch. A saved input takes precedence on a repeated attempt; its version and required invariants must still be accepted.

Define the weak prompt baseline precisely: apply every function, then accept the result if its message count equals the input count; otherwise use the input unchanged. Let the input have two messages, the second containing a restriction. One function drops that message. The next appends a new message containing only "continue." The final count is two, so the baseline accepts, but the restriction is lost. Our proposed obligation checks each function's declared structural invariant and the final prompt. Such invariants can preserve message IDs, roles, and call-result links. They still do not prove preservation of natural-language meaning.

Model and tool separation supports this boundary. The model adapter receives definitions but no local handler capability. The host first rejects input containing an unresolved local call; the separate tool phase must finish it. The adapter returns model text and proposed calls as data. Only the tool executor receives handler capabilities. An adapter that internally resolves calls would violate this interface and is excluded. This restriction is stronger than asking a model in prose not to run tools.

VI. Ownership and replay obligations

A scope requirement in a type states that construction needs a scope. It is not a linear type that makes resource handles impossible to copy. A plugin can still put a connection in a global variable or give it to an unmanaged callback. Our ownership obligation has two parts: every acquired resource has a registered release action, and every use remains within that owner's lifetime or an explicit transfer to another owner.

Define scope states as open, closing, and closed. Acquisition and task creation require open state. Acquisition registers its release action before cancellation can be delivered after success. Closing first prevents new acquisitions and tasks, then interrupts and awaits child tasks. It then attempts releases in reverse acquisition order. A release failure is recorded and does not discard later release actions. Closed state is reached only when this work ends. The host calls close in a finally path after success, build failure, hook failure, or interruption, and preserves the original exit together with cleanup failures.

Claim 3: composition preserves local ownership under non-escape. Suppose each plugin binds all resources and child tasks to the supplied scope, no handles escape, and the host awaits closure on all exits. Sequential construction preserves ownership. Every new resource extends the same owner's resource set. Partial build failure and interruption reach that owner's close operation. Induction over acquisitions shows that closure includes every release action. All releases are attempted if child joins and earlier releases terminate. Successful remote cleanup needs a further adapter guarantee. Thus the claim does not turn cooperative cancellation into a hard termination bound.

Ownership alone also does not make every finalizer use safe. Acquire resource A, then B. Reverse release closes B before A. If A's finalizer uses B, both handles remain within the same scope but the use is too late. A sufficient rule assigns each resource its acquisition index and permits its finalizer to use only itself and lower-index resources. In reverse release order, every such lower-index resource is still live. Induction over the release list proves this claim, provided no other path releases or transfers these resources. A finalizer may use its own resource before marking it closed. Dynamic dependencies need a stronger lifetime protocol; a shared owner alone does not supply it.

Background work needs an explicit transfer. Before the parent stops waiting, the host records a child identity and its new owner. Repeated attempts use that saved route. The child opens its own scope instead of borrowing handles from the parent. A final result is published only after child closure and the corresponding terminal record. This transfer protocol permits a longer lifetime without permitting an ownerless task. Full scheduling and recovery of such children remain outside this paper.

Result hooks have a different difficulty. The host reruns their ordered chain when a saved step remains open. If a later hook fails, an earlier hook may run again. Replaying saved model output does not imply that hook effects occur once.

Local idempotence is insufficient. An operation is idempotent when applying it twice has the same effect as applying it once. Consider two hooks on state (x, y). Hook A copies y into x. Hook B sets y to one. Each is idempotent alone. Starting at $(0, 0)$, the sequence $A; B$ ends at $(0, 1)$. Repeating that sequence ends at $(1, 1)$. Each hook's own law holds, but the composition is not idempotent because the hooks interfere.

Claim 4: commuting total idempotent hooks form an idempotent chain. Let each hook be a total deterministic function on the same invariant-closed state set. Suppose each function is idempotent on the whole set, and each pair commutes there. In two copies of the ordered chain, exchange adjacent commuting functions until equal functions are next to each other. Collapse each repeated pair by its idempotence

law. The remaining chain is one copy. This proves the result for any finite number of hooks. Tests at a single start state are insufficient; partial failures, outside interleavings, and writes beyond this state model are excluded. Commutation is sufficient, not necessary, as the bounded experiment below shows. We use this elementary function law as a hook obligation; we do not claim a new general algebra theorem.

We propose two replay contracts. A hook may be pure with respect to outside state and use only stable saved input. Otherwise it needs a durable key containing turn identity, step, kind, stable registration instance, and hook version. A plugin name alone is insufficient when the same plugin appears twice. Each saved instance needs a distinct stable slot, pinned with the hook list. Reordering or inserting instances on replay is a contract change. Figure 2 gives the local write protocol. Receipt lookup, comparison, write, and insertion form one serializable transaction. A unique key prevents concurrent attempts from both creating a receipt. The transaction may retry, so its body contains no outside calls.

```
apply_hook(key, saved_payload):
  transaction:
    old = receipts.read(key)
    if old exists:
      require old.payload == saved_payload
      return old.result
    result = update_local_state(saved_payload)
    receipts.insert_unique(
      key, saved_payload, result)
    return result
```

Figure 2. Proposed local replay protocol. The state write and receipt commit together.

For a remote write, the receiving service must implement the same atomic key rule. Sending a key is insufficient if the receiver ignores it. Writing a local receipt before or after an unrelated remote call leaves a crash window. If the receiver cannot give a stable result for the key, the hook exposes an unknown outcome instead of claiming safe replay.

These obligations apply to partial prefixes as well as complete hook chains. A receipt for each hook lets a successful prefix return its prior result while the failed suffix is retried. A receipt only for the whole chain cannot cover a crash halfway through it. Pin the hook list and versions when a turn is accepted, or reject replay after an incompatible change. Replay count and the current wall clock are unsuitable sources for a stable key. **Claim 5: retained atomic receipts give at most one local write per key.** Assume a stable key and payload, serializable transactions, a unique receipt constraint, atomic commit of state write and receipt, rollback of the whole transaction on abort or key conflict, and receipt retention throughout the retry period. The transaction body has no effects outside that transaction. In a serial order, let the first successful transaction create the receipt. Each later transaction reads that receipt and returns its saved result; a conflicting transaction cannot keep its state write after abort. Hence at most one state write commits for the key. The argument gives no eventual-success guarantee. Expiring a receipt while retries remain possible breaks the premise. This is a local guarantee, not an end-to-end exactly-once claim.

VII. Worked combinations and failure traces

Consider a proposed bundle with a quota check Q, a prompt filter P, a tool plugin T, and an accounting hook U. Registration runs in that order. Q returns false when a saved step budget is exhausted. P masks text without changing message positions. T registers a `write_note` definition and handler. U records usage under a stable hook key. This is an illustrative bundle, not an empirical workload.

On a denied step, construction may open T's connection first. The check runs, returns false, and the host dispatches neither the model nor `write_note`. Scope closure releases the connection. This trace satisfies dispatch denial while disproving the stronger claim that denial implies no I/O of any kind. If connection opening itself is forbidden before approval, the resource must be acquired later under a different contract.

On an allowed step, the host transforms the prompt, calls the model, saves its result, runs the tool, and saves tool output. Hook U then commits a usage record. Suppose the next hook fails. The turn remains unfinished, and replay invokes U again. In the weak baseline, U is a plain increment and doubles the record. Under Figure 2, it returns the earlier receipt and leaves the total unchanged. This is a local composition obligation; it does not require a new workflow recovery algorithm.

TABLE I
Negative combinations for validation. Well-typed interfaces can admit each case.

Combination	Failure or required condition
Two `send` tools	Right replacement may change input and behavior. Validate the pair.
Check then state writer	The later write may invalidate the checked fact.
Mask then expansion	Expansion can restore text the mask removed.
Counter then failed hook	The counter repeats unless its effect is keyed.
Scoped build then detached task	A copied handle can outlive its connection.
Filter then later registration	A later tool can escape the intended final selection.

The last case explains why tool selection belongs after complete registration. Define selection as filtering the final definition map by a fixed mode predicate, while preserving hooks. The executor must resolve names only through that selected map. In the flawed baseline, selection runs before a later plugin adds a write tool; the added tool then escapes the filter. Keeping unused handlers in a private map is harmless only when no unchecked path dispatches them. This is a dispatch obligation, not a complete access-control algorithm.

VIII. Validation method and measured finite results

We propose a layered validator. At build time, types preserve the tool map, failure unions, and service requirements. Registration validation checks selected definition-handler pairs, declared replacements, store-name conflicts, hook order,

and ownership declarations. A trace harness then checks behavior at host boundaries. Finally, review or a small proof checks semantic obligations that declarations alone cannot establish, such as a prompt transform preserving a predicate.

The proposed static checker has four rules. Only the host entry may run a computation. Every task creation must name an owner. Every resource acquisition must name a release action. An error handler must preserve the cause or send it to an explicit internal error sink before producing a public error. These are checkable rules in a small typed intermediate language whose operations are resolved names. A linter that recognizes only textual names is a weaker baseline: aliases and foreign wrappers can escape its view. Neither checker proves that a release action terminates or that a hook declaration is truthful.

The full proposed study asks which harmful combinations escape types, which need trace tests, and what validation costs. Only the finite dynamic part below has been run; the static checker and production cost study remain proposed. Compare the weak typed baseline; that baseline with the four static rules; and the proposed validator. Use the same scheduler, codecs, and simulated services. Record policies and accepted inputs so that stricter rejection is not confused with better execution.

The checked artifact is `papers/experiments/paper_15.py`. Run the repository experiment command with 30 seeds; its JSON record includes per-paper counts, seed values, and source hashes. The role corpus has allow, deny, typed failure, prompt change, tool, scoped resource, keyed hook, and unkeyed hook. Its 400 ordered lists of one to three distinct roles give 736 binary grouped variants and 1,472 model/action traces. All match a separate leaf oracle; 920 blocked traces contain no dispatch. These are measured counts, not planned sample sizes.

The extended corpus includes all lists of length zero through three over those eight roles, all such lists over six tool and prompt roles, and four distinct lists of length four. We remove the duplicate empty list. This gives 847 distinct ordered lists, 1,591 grouped variants, and 3,182 traces. Every ordered full binary grouping is used, including five groupings per four-leaf list. Repeated leaves, both empty identities, multiple tool names, token conflicts, and mask/expansion order are covered. All traces match; 244 traces reject an invalid pair before any check or dispatch. Base and extended corpora overlap, so their counts must not be added as unique cases.

The synthetic call list uses sorted selected tool names. This makes order explicit and avoids relying on map insertion order. The oracle scans leaf definitions from right to left for selection, computes the guard prefix directly, and validates selected tokens before the prefix. It calls none of `merge`, `build`, `check`, `dispatch`, or `execute`. Independence means separate algorithms over common data, not a second team or a mechanized proof. A prior one-tool corpus hid an iteration-order disagreement; the multiple-tool corpus now checks that boundary.

TABLE II
MEASURED FINITE RESULTS. FAMILIES HAVE DIFFERENT UNITS AND OVERLAPPING CASES.

Finite family	Measured size	Outcome
Base role traces	1,472	All equal the leaf oracle.
Extended role traces	3,182	All equal; 244 reject before dispatch.
Idempotent map pairs	1,681	264 chains fail idempotence.
Completed scope cases	124	All preserve exit and release failures.
Blocked scope cases	3	All remain closing.
Finalizer dependency graphs	4,166	76 safe; 4,090 rejected.

There are 256 total functions on four states. Exactly 41 are idempotent, giving 1,681 ordered pairs. Of these, 601 commute and have an idempotent chain; 264 have a non-idempotent chain; and 816 do not commute but still have an idempotent chain. This exhausts the stated four-state domain. It supports Claim 4 and shows why its sufficient condition must not be called necessary. It says nothing about a larger or infinite state space without the proof's assumptions.

The scope machine checks four exit kinds, zero through four resources, and every release-failure subset: 124 completed cases. It also checks blocked child joins and two blocked-release positions. Completed cases attempt every release, after child joins, in reverse order, preserving the original exit and each release failure. Blocked cases remain closing and reject new work. A separate live-resource oracle checks all 4,166 directed dependency graphs through four resources, excluding self-edges. Exactly 76 obey the lower-index rule; every other graph contains a use after release. These results concern a finite ledger, not actual threads or adapters.

Six single-rule ablations each run one harmful witness and one safe control. Removing pair validation permits a wrong handler; eager checks run a suffix after denial; omitting cleanup leaves a resource; count-only prompt validation loses a rule; omitting receipts repeats a write; and renewing the start time permits an expired retry. The full rules prevent all six chosen faults and accept all six controls. These are diagnostic witnesses, not estimates of a general detection rate. Receipt tests also show that two instances sharing one key lose one intended write, while distinct stable instance keys preserve both. Thirty fixed seeds produce 213 write attempts with injected pre-commit failures; each key commits once. No population confidence interval is attached to these designed cases.

IX. Further evaluation and remaining obligations

Inject failure after each resource acquisition, between hooks, after a saved model result, and before step completion. Add interruption while a hook or tool waits. Use a virtual clock for deadlines and deterministic model and tool fakes. Record dispatch after denial, mismatched definition-handler pairs, live resources after successful closure, duplicate hook writes, and prompt-invariant violations. Mutations include

reversing one precedence rule, evaluating the second check eagerly, replacing an owned task with an unowned task, and removing a replay receipt.

For finite traces, report exact counts by mutation family, missed cases, and false rejections. Do not attach a population confidence claim to exhaustive cases from a designed model. For randomized schedules, publish seeds and group failures by the shared plugin combination. Correlated traces are not independent trials. Include safe intentional replacements and commuting pure transforms to measure over-rejection. Ablations remove collision validation, replay validation, scope checks, or phase checks one at a time.

Measure cost separately with one, two, four, eight, and sixteen registrations and tool maps of zero, ten, or one hundred entries. Compare paired runs with the same seed, warm-up, and process setup. Use at least thirty fresh process repetitions per configuration, report medians and tail values, and bootstrap paired differences by process. Publish machine details, exact dependency versions, and raw timings. No such measurements are available in this paper. Finite tests cannot prove the protocol for arbitrary plugins.

X. Independent fixtures and contract generation

A reproducible test case is a data record: ordered plugin descriptors, binary grouping, initial step frame, mode predicate, virtual-clock events, and one failure cut. A descriptor lists its tool definitions and handler tokens, hook instruction lists, resource requests, and replay mode. The generator emits data rather than executable plugin closures. A small interpreter then constructs computations from that data. This permits the same corpus to drive each validation configuration.

Use bounded instructions with explicit meanings. `check(v)` records a check and returns Boolean v. `fail(e)` exits with e. `acquire(r)` registers resource r; `use(r)` requires it to remain open. `set(k,v)` replaces a local value; `add(k,v)` increments it. `copy(k,j)` reads j and writes k. Prompt instructions remove, append, mask, or expand one named message. Model output is a fixed text and call list. A tool response is a fixed value with a counter that records actual dispatch. This language expresses the counterexamples without a live model or network.

The oracle should not call the merge implementation. For each tool name, it scans the leaf descriptors from right to left and takes the first definition; it then checks the selected handler token. It computes the expected check prefix directly from the descriptors, stopping at the first false or failed check. A resource ledger checks every use against its owner's state and every completed close against its release list. A receipt ledger counts committed effects by stable key and compares full payload values. Prompt checks compare declared IDs, roles, and call-result links.

Each failure cut produces a failed execution and a new attempt from committed state. These faults unwind scopes; abrupt process loss is outside this local harness. A build retry opens a fresh scope, while result hooks keep stable keys. Finalizers finish, fail and finish, or remain blocked. Blocking tests the termination assumption, not a leak after successful closure. Publish input cases and traces. Shrinking removes plugins, instructions, and message parts while preserving the failed oracle predicate.

API consistency is a separate obligation. Define one schema graph for step input, saved result, registration error, and public diagnostic. Derive the HTTP description and client codecs from that graph, and require the server decoder to use the same versions. A diagnostic contains a stable error kind and safe public fields; an internal cause stays in a separate record linked by a correlation ID. Generation establishes common schema ancestry, not correct execution order.

If two authentication mechanisms reach the same operation, generate equivalent requests through each and compare normalized execution frames and traces. Hold identity and policy constant. Different permissions need not produce the same result. These fixtures test transport preservation of the runtime contract, not the authorization algorithm. The finite role corpus, interpreter, and results above are complete for their stated bounds. The static checker, transport-trace comparison, randomized scheduler, trace shrinking, and production measurements in this section remain further work.

XI. Limits and conclusion

The algebra abstracts from scheduler timing, compiler defects, and arbitrary foreign code. The finite experiment shares its specification with the implementation under test and can therefore share a wrong assumption. Its separate oracle and negative cases reduce some errors but do not remove this risk. No measured result establishes that a production runtime meets this proposed protocol. Associativity does not imply equal latency or fairness. Logs that expose grouping identities can differ even when tool behavior agrees. Remote termination needs an adapter guarantee beyond local scope closure.

Replay needs durable storage or a receiver with the stated key protocol. Otherwise a plugin must expose uncertainty. Prompt structure checks also cannot prove that a model will interpret two prompts alike. These limits require explicit assumptions.

Composable execution needs more than a shared plugin type. The defined registration protocol supplies a control boundary, and its ordered merge admits precise local laws. Those laws support grouping but usually forbid free reordering. Fail-stop checks, separate model and tool execution, scoped ownership, and replay contracts preserve distinct constraints under distinct assumptions. The finite artifact makes the assumptions and selected failures testable. Its bounded results, together with the conditional proofs, support precise local claims. They do not establish correctness, speed, or safe composition for an arbitrary production runtime.

References

[1] G. Plotkin and M. Pretnar, "Handlers of Algebraic Effects," in *Programming Languages and Systems*, in Lecture Notes in Computer Science, vol. 5502. Springer, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.

[2] N. Wu, T. Schrijvers, and R. Hinze, "Effect Handlers in Scope," in *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, 2014, pp. 1–12. doi: 10.1145/2633357.2633358.

[3] M. Hyland, G. Plotkin, and J. Power, “Combining Effects: Sum and Tensor,” *Theoretical Computer Science*, vol. 357, no. 1–3, pp. 70–99, 2006, doi: 10.1016/j.tcs.2006.03.013.

[4] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, 2014, doi: 10.4204/EPTCS.153.8.

[5] W. Tang and S. Lindley, “Rows and Capabilities as Modal Effects,” *Proceedings of the ACM on Programming Languages*, vol. 10, no. POPL, pp. 923–950, 2026, doi: 10.1145/3776674.

[6] J. McCall, J. Groff, D. Gregor, and K. Malawski, “Structured Concurrency.” Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/swiftlang/swift-evolution/blob/main/proposals/0304-structured-concurrency.md

[7] C. Matache, S. Lindley, S. Moss, S. Staton, N. Wu, and Z. Yang, “Scoped Effects, Scoped Operations, and Parameterized Algebraic Theories,” *ACM Transactions on Programming Languages and Systems*, vol. 47, no. 2, p. 8:1–8:33, 2025, doi: 10.1145/3731678.

[8] B. Kragl, S. Qadeer, and T. A. Henzinger, “Refinement for Structured Concurrent Programs,” in *Computer Aided Verification*, in Lecture Notes in Computer Science, vol. 12224. Springer, 2020, pp. 275–298. doi: 10.1007/978-3-030-53288-8_14.

[9] OpenAPI Initiative, “OpenAPI Specification v3.1.1.” Accessed: Oct. 27, 2026. [Online]. Available: https://spec.openapis.org/oas/v3.1.1.html